Arquitetura e gestão de mudanças

Estilos arquiteturais, MVC, gestão de configuração e tipos de manutenção.

A questão testa conhecimentos a respeito de alguns modelos arquiteturais de software, estudados em disciplinas correlatas à área de engenharia de software. Vamos, a seguir, relembrar o tema.

Arquitetura de software

Do mesmo modo como a planta baixa de uma casa é apenas uma representação do edifício físico, a representação da arquitetura de software não é o software em si. Ela é apenas uma representação, que permite:

- analisar o atendimento dos requisitos;
- considerar alternativas de arquitetura em um estágio em que fazer mudanças de projeto ainda é fácil;
- reduzir os riscos associados ao desenvolvimento do software.

A arquitetura de software oferece uma ampla visão do sistema que será construído. Ela representa a estrutura e a organização dos componentes de software, suas propriedades e as conexões entre eles. Os componentes de software incluem módulos de programas e as várias representações de dados manipuladas por eles.

De forma geral, um estilo de arquitetura fornece diretrizes gerais para o design do sistema, e um padrão de arquitetura oferece uma solução mais específica para um problema conhecido, de forma que um padrão pertença a um estilo. A seguir, vamos estudar alguns estilos de arquiteturas.

Arquiteturas monolíticas

Nas arquiteturas monolíticas, o sistema conta com baixa modularização, sendo enxergado e implementado como um único projeto. Todos os componentes de software em um sistema monolítico são interdependentes, devido aos mecanismos de troca de dados dentro do sistema.

Como exemplo, vamos imaginar um sistema para uma loja de calçados. Todos os usuários utilizarão o mesmo sistema, e será preciso que ele esteja dividido em: autenticação e perfis de usuários, compra de produtos, estoque e vendas.

No padrão monolítico, todos os módulos mencionados acima estarão em um único projeto. Se é feita uma alteração no serviço de vendas, todos os outros módulos do projeto também terão que subir novamente para o servidor junto com essa alteração.

Em tempo de execução, todos os módulos de um sistema são executados, pelo sistema operacional, como um processo único. A figura 1 ilustra um sistema de nove módulos (de M1 a M9) que segue o estilo arquitetural monolítico. Em tempo de execução, o sistema executa como um único processo, que é representado pelo quadrado que envolve os nove módulos.

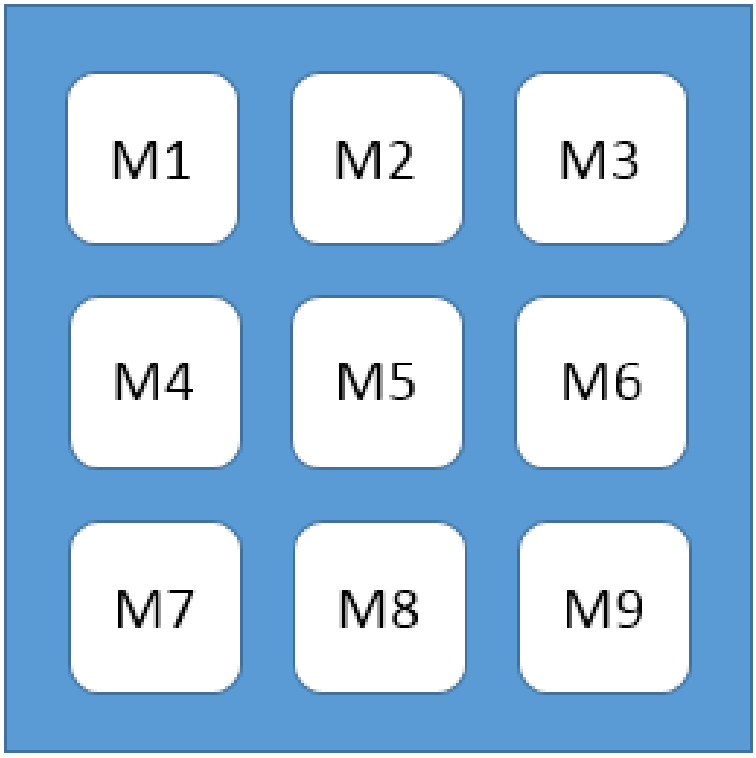


Figura 1. Sistema de nove módulos que segue o estilo arquitetural monolítico.

VALENTE, [s.d.] (com adaptações).

Em geral, um sistema monolítico é mais simples de ser desenvolvido, pois a organização fica concentrada em um único sistema. No entanto, é trabalhoso modificá-lo, pois pequenas mudanças afetam grandes áreas da base de código. Além disso, não há flexibilidade em relação à linguagem de programação: aquela que for escolhida no início do projeto terá que ser seguida para todos os módulos a serem implementados.

Arquiteturas em microsserviços

Para contornar as desvantagens dos sistemas em arquiteturas monolíticas, algumas empresas passaram a migrar os seus “monólitos” para arquiteturas baseadas em microsserviços. Nessas arquiteturas, fazemos com que certos módulos do sistema sejam executados em processos independentes, sem compartilhamento de memória entre eles. Nesse caso, o sistema é altamente modularizado não apenas em tempo de desenvolvimento, mas também em tempo de execução. Com isso, é menos provável que mudanças em um módulo causem problemas em outro.

O nome desse estilo de arquitetura refere-se a cada módulo como um serviço. Além disso, os serviços são “micro” porque cada um deles implementa apenas uma funcionalidade simples.

Na figura 2, podemos ver o diagrama que ilustra um sistema de nove módulos implementados por meio de uma arquitetura em microsserviços. Esses módulos são executados por seis processos independentes, representados pelos retângulos externos aos módulos. Os módulos M1, M2, M3 e M6 são executados individualmente, ou seja, cada um em um processo independente. Os módulos M4 e M5 são executados em dupla. Por fim, os módulos M7, M8 e M9 são executados, em conjunto, em um processo.

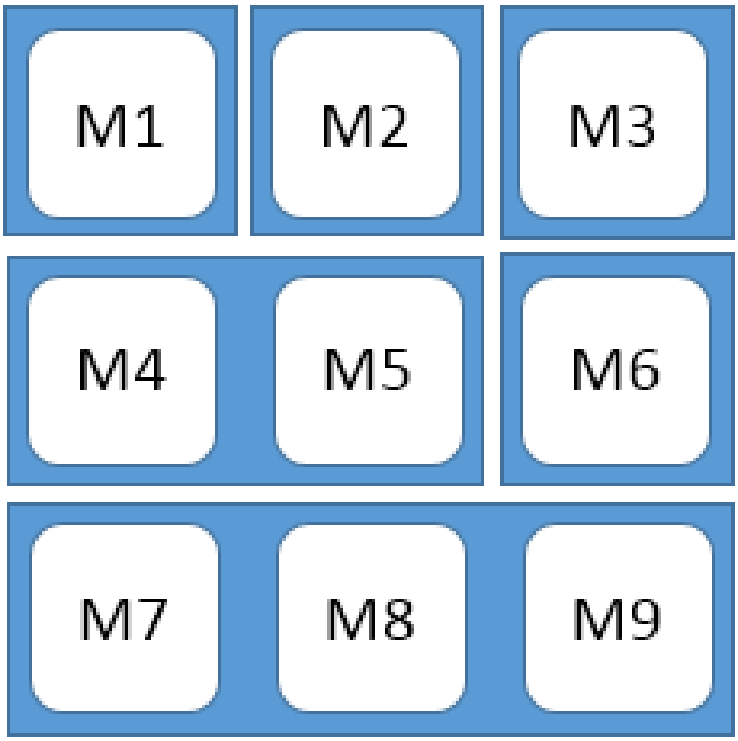


Figura 2. Sistema de nove módulos que segue o estilo arquitetural em microsserviços.

VALENTE, [s.d.] (com adaptações).

Arquiteturas orientadas a objetos

Nas arquiteturas orientadas a objetos, os componentes de um sistema encapsulam dados e as operações que devem ser aplicadas para manipular esses dados. A comunicação entre componentes é realizada por meio da passagem de mensagens.

Arquiteturas orientadas a objetos utilizam os princípios e os conceitos da programação orientada a objetos. Cada objeto tem uma interface definida pelos seus métodos, e a comunicação ocorre por meio da chamada desses métodos.

Padrão Modelo-Visão-Controle

O padrão arquitetural Modelo-Visão-Controle (MVC, do termo em inglês Model-View- Controller) pode ser classificado como um padrão de arquitetura orientado a objetos, que define que as classes de um sistema devem ser organizadas em três grupos, elencados a seguir.

- Visão: grupo de classes responsáveis pela apresentação da interface gráfica do sistema.
- Controle: conjunto de classes que tratam e interpretam eventos gerados por dispositivos de entrada, como o teclado ou o mouse. Como resultado da interpretação desses eventos, as classes de controle podem solicitar uma alteração no estado do modelo ou da visão.
- Modelo: grupo de classes que armazenam os dados manipulados pela aplicação e que têm a ver com o domínio do sistema, ou seja, com a lógica do negócio.

O relacionamento entre esses grupos de classes pode ser exemplificado conforme exposto na figura 3, que ilustra um sistema construído no modelo MVC.

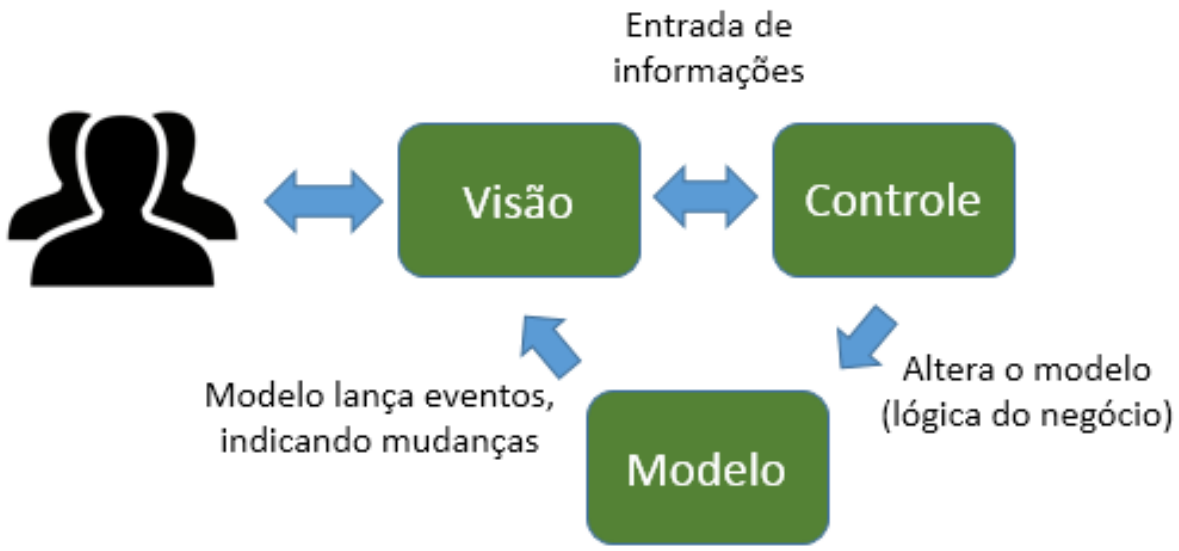


Figura 3. Fluxo de eventos e de informações em um sistema construído em modelo MVC.

UNIVERSIDADE FEDERAL DE CAMPINA GRANDE, [s.d.]. Disponível em <http://www.dsc.ufcg.edu.br/~jacques/cursos/map/html/arqu/mvc/mvc.htm >. Acesso em 19 abr. 2026 (com adaptações).

A questão testa conhecimentos a respeito das atividades envolvidas na gestão de configuração de software. É importante frisar que essas atividades podem ser especificadas de diferentes formas por diferentes autores da área de engenharia de software. A nomenclatura presente nas alternativas da questão corresponde à utilizada por Roger Pressman e Bruce Maxim, no livro intitulado “Engenharia de Software: uma abordagem profissional”. Vamos, a seguir, estudar um pouco desse conteúdo.

Gestão de configuração de software

Na construção de um sistema computacional, alterações são inevitáveis. Essas mudanças podem causar confusão quando não são analisadas antes de serem implementadas, ou quando não há boa comunicação entre os membros da equipe responsável pelo projeto. À medida que o trabalho de engenharia de software avança, a equipe de software cria uma hierarquia de itens de configuração de software (SCIs, do termo em inglês software configuration items). Um SCI é uma entidade única identificável, que é gerenciada e documentada durante o ciclo de vida de um projeto. Essas entidades podem incluir códigos- fonte, documentos, scripts e outros elementos relacionados ao sistema. Os SCIs são chamados coletivamente de configuração de software.

Nesse cenário, a gestão de configuração de software (SCM, do termo em inglês software configuration management) é responsável por identificar, organizar e controlar modificações ao longo do clico de vida do software. O objetivo da SCM é maximizar a produtividade e minimizar possíveis erros.

O processo de gestão de alterações de software define uma série de tarefas que têm quatro objetivos principais:

- identificar todos os itens que definem a configuração do software;
- gerenciar alterações de um ou mais desses itens;
- facilitar a construção de diferentes versões de uma aplicação; e
- assegurar que a qualidade do software seja mantida conforme a configuração evolui. Com o intuito de atingir esses objetivos, são definidas cinco tarefas SCM: identificação, controle de versão, controle de alteração, auditoria de configuração e elaboração de relatórios. Essas

tarefas são mostradas no diagrama da figura 1. Conforme a disposição da figura, as tarefas SCM podem ser vistas como camadas. Os SCIs “fluem” para fora através dessas camadas por toda a sua vida útil, tornando-se, finalmente, parte da configuração do software de uma ou de várias versões do sistema de software.



Figura 1. Camadas do processo SCM.

PRESSMAN e MAXIM, 2021 (com adaptações).

A atividade de identificação envolve a identificação clara dos SCIs que estão sujeitos à configuração. A gestão de configuração requer a capacidade de identificar e de rastrear esses elementos específicos dentro do sistema.

A atividade de controle de alterações pode combinar procedimentos humanos e ferramentas automatizadas, proporcionando um mecanismo para o controle de alterações. Resumidamente, nessa atividade, uma solicitação de alteração é apresentada e avaliada. Os resultados da avaliação são apresentados como um relatório de alterações e, por fim, uma ordem de alteração é gerada para cada alteração aprovada.

Na atividade de controle de versão, é criada uma atualização do arquivo original logo que a alteração for feita. Alternativamente, os objetos a serem alterados podem ser “retirados” do banco de dados do projeto e, assim que a alteração é feita, os objetos são então “colocados” novamente no banco de dados, e são usados mecanismos de controle de versão apropriados para gerar a próxima versão do software. O controle de versão permite o acompanhamento das alterações feitas, facilitando o trabalho colaborativo e a gestão de diferentes versões do software.

A auditoria de configuração complementa a revisão técnica, avaliando o objeto de configuração quanto a características que, em geral, não são consideradas durante a revisão. A revisão técnica focaliza a exatidão técnica do objeto de configuração modificado. Os revisores avaliam o SCI para determinar a consistência com outros SCIs, omissões ou efeitos colaterais em potencial. A auditoria entra como complemento, verificando se a configuração do software está em conformidade com as políticas, com os padrões e com os requisitos estabelecidos.

É produzido, também, o relatório de status de configuração, que reporta o que aconteceu no processo de modificações, quem fez as alterações, quando elas foram feitas e o que será afetado por elas. O relatório não necessariamente é gerado ao final do processo. Ele pode ser gerado em diferentes fases do desenvolvimento, para informar sobre o estado da configuração.

Tipos de manutenção de software

Sistemas de software legados são produtos em obsolescência que, normalmente, estão sendo utilizados por uma companhia há muitos anos. No geral, eles são desenvolvidos para ter um longo ciclo de vida. Naturalmente, eles requerem modificações ao longo do tempo. Nesse contexto, a manutenção de um sistema de software legado consiste nos processos de atualização, de correção e de prevenção de problemas em um sistema de software, com o intuito de manter o software operacional no ambiente do usuário. As atividades de manutenção são tipicamente gerenciadas pelo processo de gestão de configuração de software (SCM), que administra as alterações do sistema ao longo de todo o seu ciclo de vida. Existem quatro tipos principais de manutenção de software, elencados a seguir.

- Corretiva: este tipo de manutenção é realizado para corrigir erros existentes no software, que não foram identificados ao longo do processo de testes. A correção de bugs, de erros de lógica e de problemas de desempenho são exemplos de manutenções corretivas.
- Adaptativa: esta manutenção tem como intuito adaptar o software a novas condições de trabalho, como inseri-lo em um novo ambiente ou fazê-lo funcionar sob novos requisitos de sistema. É exemplo de manutenções adaptativas a tarefa de tornar um software compatível com um novo sistema operacional ou com um novo hardware.
- Perfectiva (ou evolutiva): a manutenção perfectiva tem como foco a adição de novas funcionalidades ao sistema, assim como a melhoria das funcionalidades existentes. Isso inclui a implementação de novos requisitos ou de novas funcionalidades, que não estavam previstas no projeto original.
- Preventiva: este tipo de manutenção tem como principal objetivo melhorar a confiabilidade do sistema. Esse tipo de manutenção procura tratar situações que poderão causar erros no software, de maneira a evitar que problemas aconteçam no futuro. Em média, 50% do tempo dos desenvolvedores costuma ser dedicado à manutenção perfectiva, enquanto as manutenções adaptativa, corretiva e preventiva ocupam, respectivamente, 25%, 21% e 4%. Podemos observar essa distribuição de tempo no gráfico da figura 1.

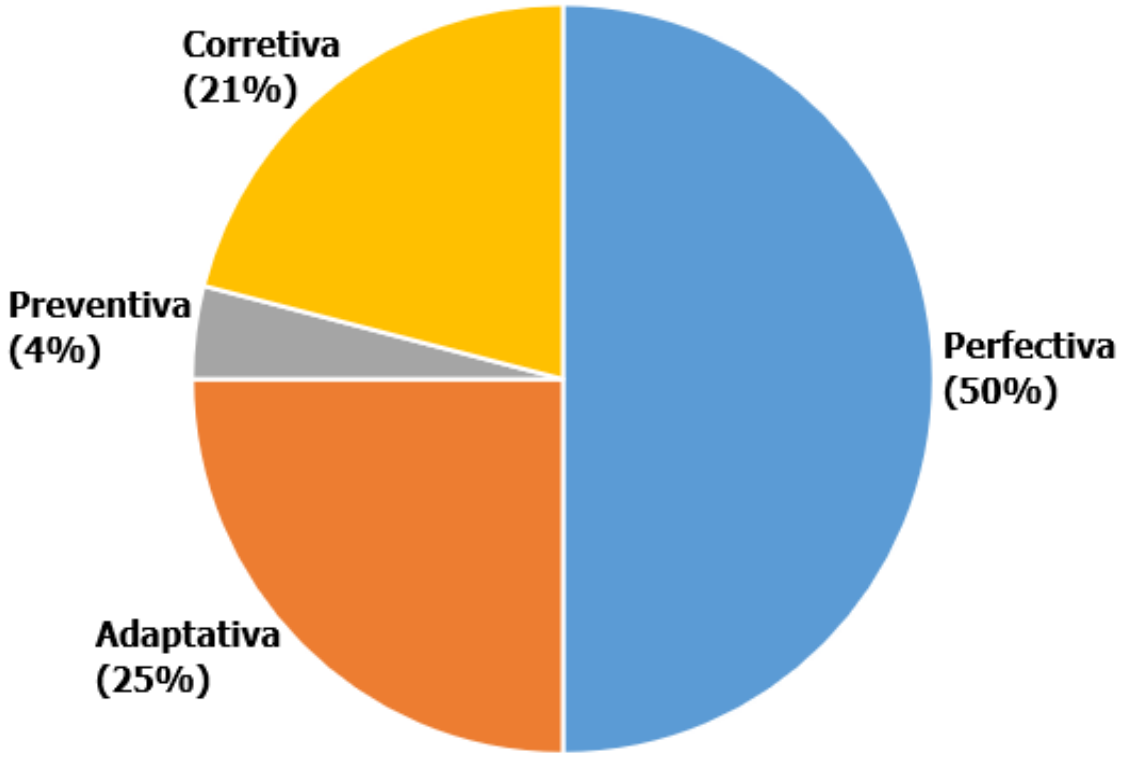


Figura 1. Distribuição do esforço de manutenção.

PRESSMAN e MAXIM, 2021 (com adaptações).

Fonte: Material Instrucional Específico – Análise e Desenvolvimento de Sistemas – Tomos 14 e 18 – CQA/UNIP (introduções teóricas). Texto e figuras reproduzidos do material original.